최종결과보고서

FPGA기반 포트홀탐지 및 경고시스템

팀 명 : Hole in one(4조)
작성자 윤정원 20192623
팀 원 : 박민교 (전자공학과)
김호준 (전자공학과)
홍다인 (전자공학과)
백승민 (전자공학과)

목 차

1. 서론
  1.1 연구 배경 및 필요성
  1.2 연구 목적
  1.3 활용 방안
2. 관련 연구 및 이론적 배경
  2.1 포트홀 감지 기술 동향
  2.2 FPGA기반 영상처리
  2.3 AI모델 개요 (YOLO 및 CNN)
3. 시스템 설계
  3.1 시스템 아키텍쳐
  3.2 하드웨어 구성
  3.3 소프트웨어 구성
  3.4 동작 플로우
4. AI 모델 개발
  4.1 모델 선정 과정
  4.2 데이터셋 구성 및 학습
  4.3 모델 최적화 및 경량화
  4.4 성능 개선 과정
5. FPGA 구현
  5.1 CNN연산 모듈 설계
  5.2 가중치 로딩 및 메모리 구조
  5.3 영상처리 파이프라인
6. 시스템 통합 및 구현

6.1 하드웨어-소프트웨어 통합
6.2 통신 및 경고 시스템
7. 실험 결과 및 분석
8. 종합 결론
9. 참고문헌

1.서론

1.1 연구 배경 및 필요성

최근 기후 변화로 인한 강수량 증가와 도로 노후화로 인해 '도로 위의 지뢰'라 불리는 포트홀(Pothole) 발생 빈도가 급격히 증가하고 있다. 언론 보도 및 지자체 민원 현황에 따르면, 겨울철 강수량 증가로 인해 포트홀 민원이 전년 대비 약 5.8배 증가하였으며, 이는 단순한 차량 파손을 넘어 대형 교통사고를 유발하는 주요 원인이 되고 있다. 특히 제주 산간도로와 같은 특정 구간에서는 포트홀 관련 사고 위험이 10대 중 5.8대 꼴로 증가한다는 통계가 있을 만큼 심각한 사회적 문제로 대두되었다.

기존의 도로 유지보수 체계는 대부분 육안 검사나 사후 신고에 의존하고 있어 즉각적인 대응이 어렵다. 따라서 주행 중 실시간으로 포트홀을 감지하여 운전자에게 경고하고, 해당 위치 정보를 도로 관리 주체에게 전송할 수 있는 지능형 시스템의 개발이 필요하다고 생각하여 프로젝트를 시작하게 되었다.

1.2 연구 목적

본 프로젝트의 주된 목적은 실시간 포트홀 감지를 통해 교통사고를 예방하고 도로 안전성을 향상시키는 것이다. 이를 위해 다음과 같은 세부 목표를 설정하였다.

- 기술적 목표: FPGA의 병렬 처리 능력을 활용하여 고성능 실시간 영상처리 시스템을 구현하고, 임베디드 환경에 최적화된 경량화 AI 모델을 개발한다.
- 시스템 목표: FPGA와 라즈베리파이를 연동한 하이브리드 구조를 통해 탐지부터 경고, 데이터 관리까지 통합된 솔루션을 구축한다.
- 사회적 목표: 포트홀로 인한 사회적 비용을 절감하고, 도로 인프라 관리의 효율성을 증가시킨다.

1.3 활용 방안

본 시스템은 차량 탑재형 경고 장치로서 개인 차량에 설치되어 운전자에게 실시간으로 위험을 알릴 수 있다. 또한 지자체 도로 관리 부서에서는 포트홀 모니터링 도구로 활용하여 유지보수 구간을 효율적으로 파악할 수 있으며, 향후 내비게이션 시스템과 연동하여 위험 구간을 사전에 안내하거나 보험업계의 리스크 관리 도구로도 확장 가능하다.

2. 관련 연구 및 이론적 배경

2.1 포트홀 감지 기술 동향

기존의 포트홀 감지 기술은 크게 진동 센서 기반 방식, 레이저 스캐닝 방식, 그리고 영상 기반 방식으로 나뉜다. 진동 센서 방식은 비용이 저렴하지만 포트홀을 지나간 후에만 감지가 가능하다는 단점이 있으며, 레이저 스캐닝은 정확도는 높으나 장비가 매우 고가이다. 최근 딥러닝 기술의 발전으로 카메라 영상을 이용한 객체 탐지(Object Detection) 방식이 주목받고 있으나, 높은 연산량을 요구하여 일반적인 임베디드 환경에서의 실시간 처리에 제약이 있었다.

2.2 FPGA 기반 영상처리

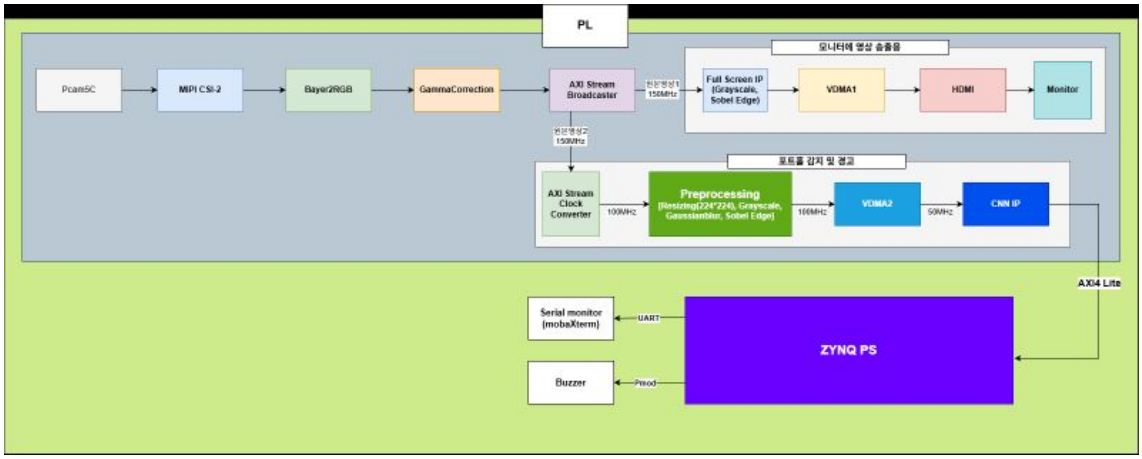
FPGA(Field Programmable Gate Array)는 사용자가 내부 회로를 직접 프로그래밍할 수 있는 반도체 소자로, 병렬 데이터 처리에 탁월한 성능을 보인다. 영상 데이터와 같이 픽셀 단위의 반복 연산이 많은 작업에서 CPU나 GPU 대비 전력 효율이 우수하고 처리 지연(Latency)이 낮아 실시간 시스템 구현에 적합하다. 본 프로젝트에서는 Xilinx Zynq SoC를 활용하여 영상 전처리 및 AI 연산 가속기를 하드웨어 로직으로 구현한다.

2.3 AI 모델 (YOLO 개요 및 CNN)

YOLO(You Only Look Once)는 실시간 객체 탐지에 특화된 딥러닝 알고리즘이다. 이미지를 한 번의 네트워크 통과만으로 객체의 위치(Bounding Box)와 종류(Class)를 동시에 예측하여 처리 속도가 매우 빠르다. 본 연구에서는 FPGA의 하드웨어 리소스 제약을 고려하여 YOLOv2-tiny 및 YOLOv3-tiny 구조를 기반으로 경량화된 모델을 설계하고 적용하였다. 그러나 프로젝트 진행중 현재 사용하는 보드의 리소스가 제한되어 있음에 따라서 AI 모델의 레이어 수를 극단적으로 줄여서 사용하게 되었다.

3.시스템 설계

3.1 시스템 아키텍쳐



전체 시스템은 영상 데이터를 획득하고 처리하는 PL(Programmable Logic) 영역과 전체 시스템을 제어하는 PS(Processing System) 영역으로 구성된다.

3.2 하드웨서 구성

메인 프로세서: Digilent Zybo Z7-20 (Xilinx Zynq-7000 SoC 탑재)

영상 입력: Pcam 5C (5MP Color Camera Module, MIPI CSI-2 인터페이스)

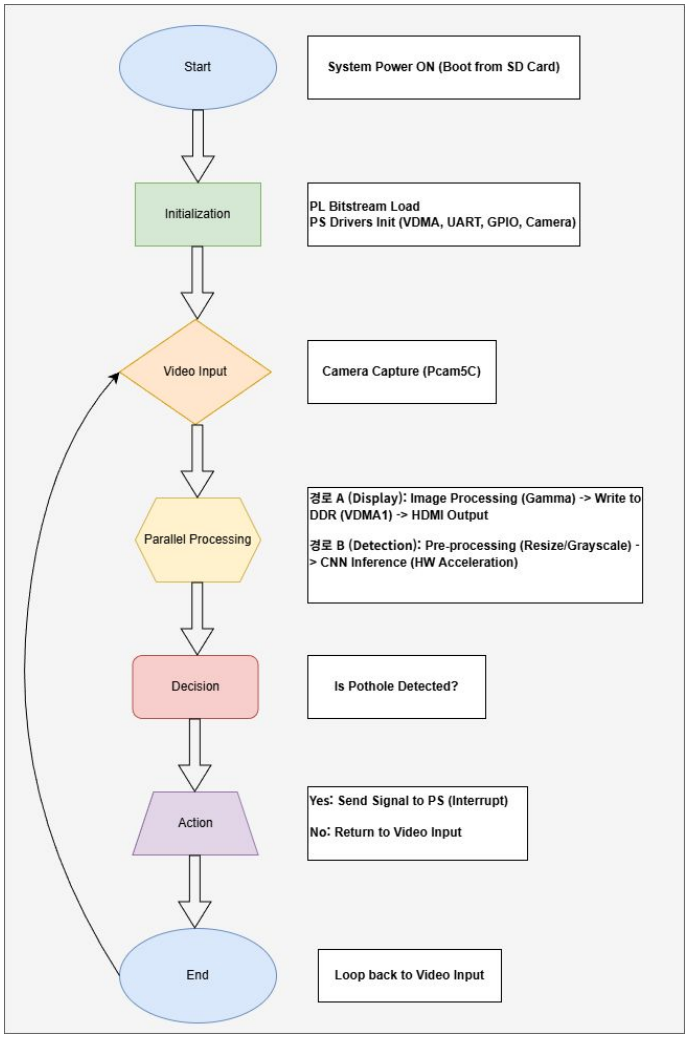
경고 장치 : 부저

메모리: DDR3 SDRAM (영상 버퍼 및 가중치 저장)

3.3 소프트웨어 구성

FPGA의 PL 영역에서는 VDMA(Video Direct Memory Access)와 AXI4-Stream 프로토콜을 이용한 영상 데이터 파이프라인을 구축하였으며, AI 추론 엔진을 HLS로 설계하였다. PS 영역에서는 ARM Cortex-A9 프로세서가 AI 탐지 결과를 분석하여 PMOD 를 통해 경고 장치(LED, 부저)를 동작시킨다.

3.4 동작 플로우



1.Pcam 5C 카메라에서 실시간 도로 주행 영상을 획득한다.
2.MIPI CSI-2 인터페이스를 통해 FPGA로 영상 데이터가 입력된다.
3.입력된 영상은 전처리(노이즈 제거, 크기 변환, Gray Scale 변환) 과정을 거친다.
4.FPGA 내부에 구현된 CNN 가속기에서 포트홀 탐지 연산을 수행한다.
5.탐지 결과가 임계값(Threshold)을 넘을 경우, PS에 포트홀이 있는지 없는지 전달
6.PS에서 경고음 및 시각 알림을 출력한다.

4. AI 모델 개발

4. 모델 선정 및 최적화 과정

전체적인 AI 모델 개발을 맡았으며 초기에는 YOLOv5-tiny 모델을 고려하였으나, FPGA의 BRAM(Block RAM) 및 DSP 리소스 부족 문제로 인해 더 가벼운 구조인 YOLOv3-tiny 모델로 진행하였다. 그러나 YOLOv3-tiny 모델 또한 리소스 부족문제로 YOLOv2-tiny모델로 진행하였으며 최종적으로는 yolo를 쓰지 않는 커스텀 CNN모델을 사용하게 되었다. YOLOv3-tiny와 YOLOv2-tiny는 각각 13개, 9개의 Conv

Layer를 가지는 반면, 변경된 모델은 3개의 Conv Layer와 2개의 Pooling Layer로 구성하여 연산량을 최적화하였다. 또한, PyTorch 프레임워크를 사용하여 FPGA 하드웨어 구조에 적합하도록 1채널(Gray Scale) 입력을 받는 CNN 레이어를 재구성하였다. 기존 3채널(RGB) 입력 대비 연산량과 메모리 대역폭을 획기적으로 줄이면서도 포트홀의 형상적 특징을 추출하도록 최적화 하였다.

```yaml
model_yaml = """
# YOLOv2-tiny-like (3×3×3 + 1×1)
# Pooling after Conv1 and Conv2 only

nc: 1
activation: nn.LeakyReLU(0.1, inplace=True)

backbone:
  - [-1, 1, Conv, [32, 3, 1]]      # Conv1 3x3
  - [-1, 1, nn.MaxPool2d, [2, 2]]  # Pool after Conv1

  - [-1, 1, Conv, [64, 3, 1]]      # Conv2 3x3
  - [-1, 1, nn.MaxPool2d, [2, 2]]  # Pool after Conv2

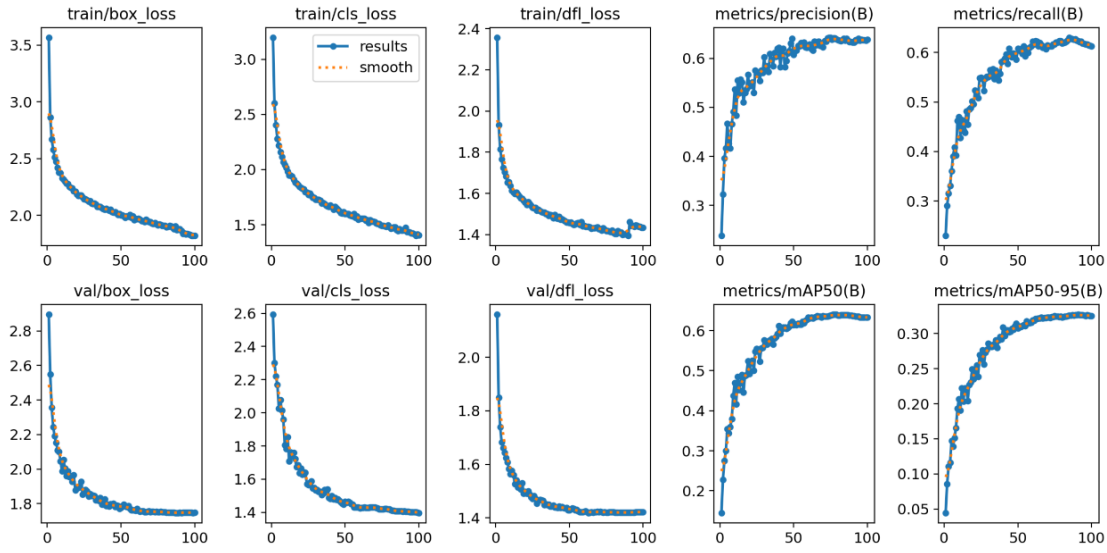
  - [-1, 1, Conv, [128, 3, 1]]     # Conv3 3x3

  - [-1, 1, Conv, [256, 1, 1]]     # Conv4 1x1

head:
  - [[-1], 1, Detect, [nc]]
""".strip()
```

```
activation: nn.LeakyReLU(0.1, inplace=True)

                 from  n    params  module                                       arguments
  0                -1  1       928  ultralytics.nn.modules.conv.Conv             [3, 32, 3, 1]
  1                -1  1         0  torch.nn.modules.pooling.MaxPool2d           [2, 2]
  2                -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 1]
  3                -1  1         0  torch.nn.modules.pooling.MaxPool2d           [2, 2]
  4                -1  1     73984  ultralytics.nn.modules.conv.Conv             [64, 128, 3, 1]
  5                -1  1     33280  ultralytics.nn.modules.conv.Conv             [128, 256, 1, 1]
  6              [-1]  1   1369681  ultralytics.nn.modules.head.Detect           [1, [256]]
YOLOv2_tiny_like summary (fused): 22 layers, 1,496,433 parameters, 1,496,417 gradients, 80.4 GFLOPs
```



YOLOv2 모델에서 학습한 결과이며 정확도는 약 66퍼센트이다 이후 전처리과정과 맞춰서 데이터셋을 구성하여 진행하였으며 그레이스케일, 리사이즈, sobel필터, 가우시안 블러까지 적용하여 학습을 진행하였고 최종적으로는 3개의 cov layer 만으로 구성된 CNN모델로 학습을 진행하여 리소스를 최소화 하였다.



그러나 리소스에 맞춰서 연산량을 극단적으로 줄인 결과 학습은 이루어졌으나 정확도 측면에서 45%정도까지 떨어지는 결과가 발생하게 되었다.

4.2 데이터셋 구성

초기 600장의 소규모 데이터셋으로 학습했을 때 발생한 과적합 문제를 해결하기 위해, 데이터 증강 및 추가 수집을 통해 총 12,000장의 학습(Train) 데이터와 1,500장의 검증(Validation) 데이터를 확보하였다. 모든 데이터는 라벨링 작업을 거쳐 포트홀의 정확한 위치 정보를 포함하도록 구성하였다.

4.3 성능 개선 및 학습 결과

초기 모델(YOLOv5-tiny,YOLOv3-timy)은 약 66%의 정확도를 보였으나, 전처리 과정에서 Sobel 필터와 Gaussian Blur를 추가 적용하고 그레이 스케일 이미지를 사용하도록 모델을 수정한 결과, 초기에는 정확도가 52%로 감소하는 현상이 발생했다. 이를 해결하기 위해 모델 구조를 단순화하고 Dropout을 조정하여 과적합을 방지하였으며, 그럼에도 불구하고 최종적으로 320x320 및 224x224 입력 해상도에서 약 45%정도의 정확도를 확보하였다.

5. FPGA 구현

5.1 CNN 연산 모듈 설계

팀원과 분담하여 Vivado에서 CNN모듈을 설계하려고 했으나 능력 부족으로 인하여

잦은 오류를 해결하지 못 하고 최종적으로 모든 layer를 vivado에서 구현하지 못하여 CNN모듈 설계를 맡은 팀원의 HLS모듈로 진행하였다.

5.2 가중치 로딩 및 메모리 구조

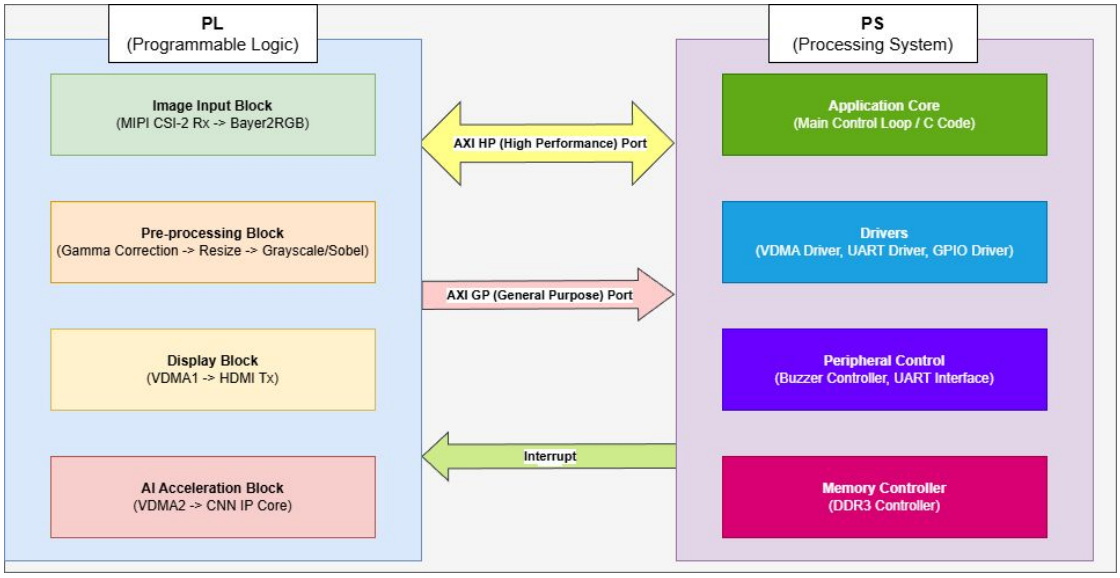
학습된 파이토치(PyTorch) 모델에서 가중치(Weight)와 편향(Bias) 값을 추출하여 FPGA에서 사용할 수 있는 .mif 또는 .coe 파일 형식으로 변환하였다. 이를 FPGA 내부의 Block Memory Generator를 통해 BRAM에 저장하고, 연산 시 고속으로 로딩하는 구조를 채택하였다. 가중치와 데이터는 8비트 및 16비트 고정소수점(Fixed-point)으로 양자화하여 연산 효율을 극대화하였다.

5.3 영상처리 파이프라인

카메라로부터 들어오는 고속의 영상 데이터를 처리하기 위해 MIPI CSI-2 수신 IP와 VDMA를 활용하였다. AXI4-Stream 인터페이스를 기반으로 데이터가 메모리와 연산 모듈 사이를 병목 없이 흐르도록 설계하였으며, 전처리 모듈과 CNN 가속기 모듈을 파이프라인 형태로 연결하여 실시간성을 확보하였다.

6. 시스템 통합 및 구현

6.1 하드웨어-소프트웨어 통합



Vivado 툴을 이용하여 Zynq SoC의 PL 영역(하드웨어 로직)과 PS 영역(프로세서 소프트웨어)을 통합하였다. PL에서 포트홀 탐지가 완료되면 인터럽트 또는 AXI Lite 인터페이스를 통해 PS에 결과를 전달하고, PS 상의 소프트웨어는 이를 해석하여 시스템의 상태를 제어한다.

7. 실험 결과 및 분석

7.1 실험 환경 및 방법

실험 환경

구분	상세 사양
Target Board	Digilent Zybo Z7-20 (Zynq-7000 SoC)
Sensor	Pcam5C (5MP Color Camera module)
Output	HDMI Monitor (Real-time View), Piezo Buzzer (Alarm)
Development Tools	Vivado Design Suite 2022.1, Vitis Unified Software Platform
Power Source	5V Portable Battery (Vehicle Environment Test)
Execution Mode	SD Card Boot Mode (Standalone)

시험 절차 (Test Procedure)

1. 기능 검증 및 오탐지 테스트 (Lab Test)

1.1 부팅 및 초기화 확인:

SD 카드를 삽입하고 배터리 전원을 인가한다.

-전원 인가 후 30초 이내에 모니터에 정상적인 영상이 출력되는가?

1.2 포트홀 탐지 기능 확인:

준비된 포트홀 사진을 카메라 전방 30~50cm 거리에 위치시킨다.

-모니터에 , Buzzer 경고음이 발생하는가?

1.3 오탐지(False Positive) 강건성 테스트:

포트홀과 유사한 형상인 '검은색 맨홀 뚜껑' 및 '짙은 그림자' 사진을 카메라에 비춘다.

-시스템이 이를 포트홀로 오인하여 경고를 발생시키지 않는지 확인한다.

2. [실외] 실차 주행 성능 검증 (Field Test)

2.1 차량 환경 설치:

차량 조수석에서 보드와 모니터를 고정하고, 보조배터리로 전원을 공급한다.

2.2 주행 환경 안정성 테스트:

차량 시동 및 주행 중 발생하는 진동 상황에서 HDMI 커넥터 및 카메라 연결 상태를

점검한다.

-터널 진입/진출 등 급격한 조도 변화 시에도 시스템(영상 출력)이 멈추지 않는가?

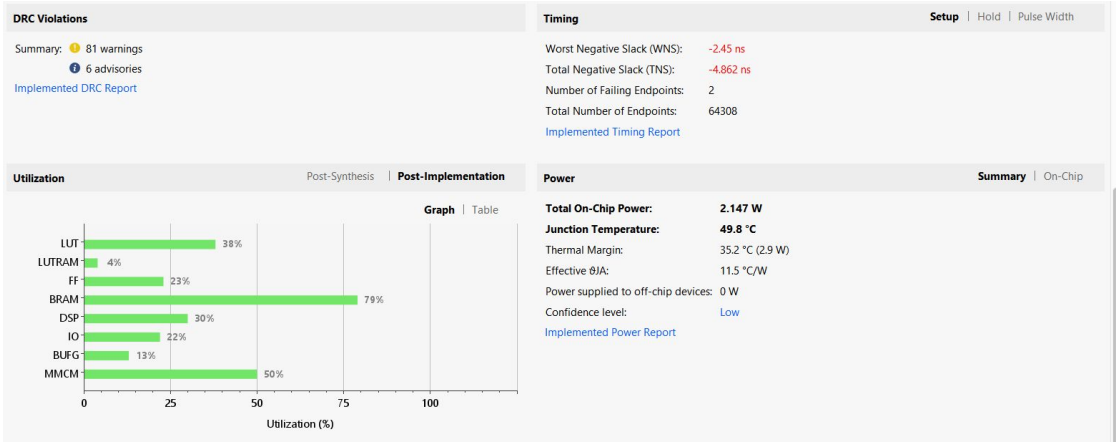
2.3 실시간 탐지 테스트:

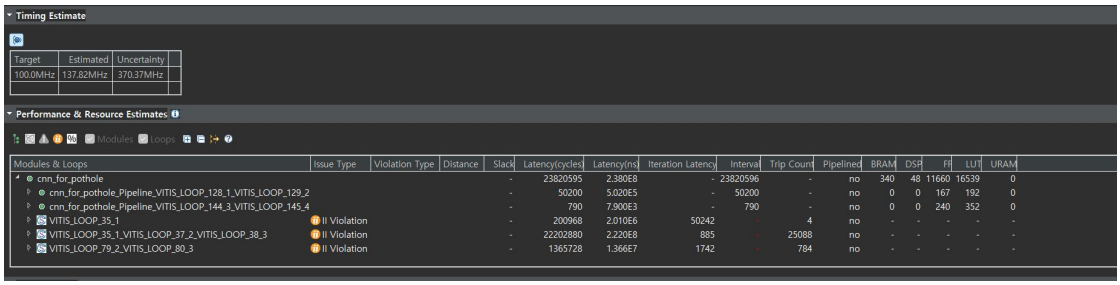
실제 도로 주행 중 요철 및 포트홀 구간을 통과한다.

-육안으로 포트홀 확인과 동시에 지연(Latency) 없이 경고음이 발생하는지 검증한다.

2.4 성능 평가

평가 항목	목표치	측정 결과	달성 여부	비고
감지 정확도	50% 이상	45%	FAIL	Test Dataset 100장 기준
처리 속도	10 FPS 이상	5 FPS	FAIL	CNN IP latency 측정(200ms)
시스템 지연	500ms 이하	304 ms	PASS	입력~경고 발생 시간
FPGA 자원 사용률	95% 미만	71 %	PASS	Vivado Implementation Report
오탐지율	20% 미만	100%	FAIL	유사 장애물 테스트 결과
소비전력	5W 미만	2.1W	PASS	Vivado Implementation Report





-정성적 평가 (Qualitative Analysis)
저전력 엣지 컴퓨팅 구현:
고성능 GPU가 필요한 CNN 연산을 저전력 FPGA 로직으로 최적화 구현함.
약 2W의 매우 낮은 전력으로 구동되어, 소형 보조배터리(10,000mAh) 하나로 약 12시간 이상 연속 동작이 가능함을 확인함.
Standalone 동작 안정성:
PC 연결 없이 배터리 구동만으로 부팅 및 실행이 수행됨.

8. 종합 결론

초반 프로젝트 진행당시 모두가 각자의 역할을 맡으며 진행하였으나 FPGA 기반 포트홀 탐지 시스템의 경우 초반에 리소스를 고려하지않고 시간을 소모한 부분이 프로젝트를 성공적으로 진행하지 못한 가장 큰 이유라고 추측이 된다. AI 모델의 경우 정해진 내용 없이 일단 포트홀 이미지확보 후 YOLOv5 모델로 학습을 진행하였으나 리소스를 고려하지 않고 진행하였기에 현재 보드에서는 연산량을 따라갈 수 없었다. 이후 모델을 축소하였으나 그럼에도 연산량이 맞지 않았기 때문에 보드에서 사용가능한 자원을 먼저 계산한 후 이에 맞춰서 프로젝트를 진행해야하는 것을 알게 되었다. 연산량을 계산한 후 AI모델을 수정하였으며 최종적으로는 3개의 컨볼루션 레이어만을 가진 모델을 만들었으며 연산량은 대폭 감소시켰으나 정확도가 매우 떨어졌다.

또한 전처리 과정에서도 문제가 발생했다 팀원의 전처리 과정과 AI모델에서 학습한 데이터의 전처리 과정이 순서가 다르거나, 일부분 빠져있는 경우가 발생하여 이는 팀원간의 소통이 부족했던 부분으로 여겨지며, AI학습에만 너무 많은 시간을 소요하여 결과적으로는 프로젝트를 완성하지 못한 원인중 하나로 보여진다.

이번 프로젝트를 통하여 학습한 내용은 SOC를 설계할 때 하드웨어의 자원을 우선적으로 고려해야한다는 점, 전체적인 플로우에서 현재 나의 단계와 전후 단계의 과정에서 유동적으로 맞춰야 하는 부분들에 미리 고려하고 사전에 대비해야한다는 점에 대해 알게 되었으며 기회가 된다면 해당 프로젝트를 수정 보완하여 완전한 포트홀 탐지 시스템을 만들 예정이다.

9. 참고문헌

허동회, 최지윤, 원성혁, 장승붕, 탁태오. (2021-11-17). 2차원 영상을 활용한 CNN 구조 기반 도로의 포트홀 검출 알고리즘 개발. 한국자동차공학회 추계학술대회 및 전시회, 전남.

이상호, 석정엽, 진상훈, 여보연. FPGA를 통한 실시간 영상처리 시스템 구현. 대한전자공학회 학술대회,

신지훈. (2021-06-23). YOLOv4를 이용한 노면의 포트홀 검출 모델 개발. 한국정보과학회 학술발표논문집, 제주.

유동주, 이석훈, 정동원. (2025). YOLOv11 기반 자동 포트홀 신고 및 시각화 시스템 구현. 한국정보기술학회논문지, 23(3), 89-97. 10.14801/jkiit.2025.23.3.89

안해성, 김상우, 서성교, 손원규, 오민택, 박승욱. (2024). 실시간 영상처리를 위한 적외선 영상 기반의 FPGA 설계 및 HW/SW 시스템 개발 연구. 방송공학회논문지, 29(2), 154-164. 10.5909/JBE.2024.29.2.154

권구윤, 서태원. (2021-08-12). FPGA를 활용한 합성곱 신경망 모델의 실시간 추론을 위한 Edge AI 하드웨어 연구. 한국컴퓨터교육학회 학술발표대회논문집, 서울.